

Dynamic Programming $\Rightarrow$

"Those who forget the fast are condemned to repeat it."

- ① Recursion
- ② Recursion + Memoization
- ③ Recursion + Tabulation
- ④ Space Optimization

$$12345 \rightarrow 1+2+3+4+5 = 15$$

$$ans = 0 + 5$$

Sum of digits

$$12345 \div 10 = 5$$

$$12345 / 10 = 1234$$

$$(\%10)(/10)$$

```
(n == 0)
    return 0;
```

```
return (n%10) + func(n/10);
```

$$5 + f(1234)$$

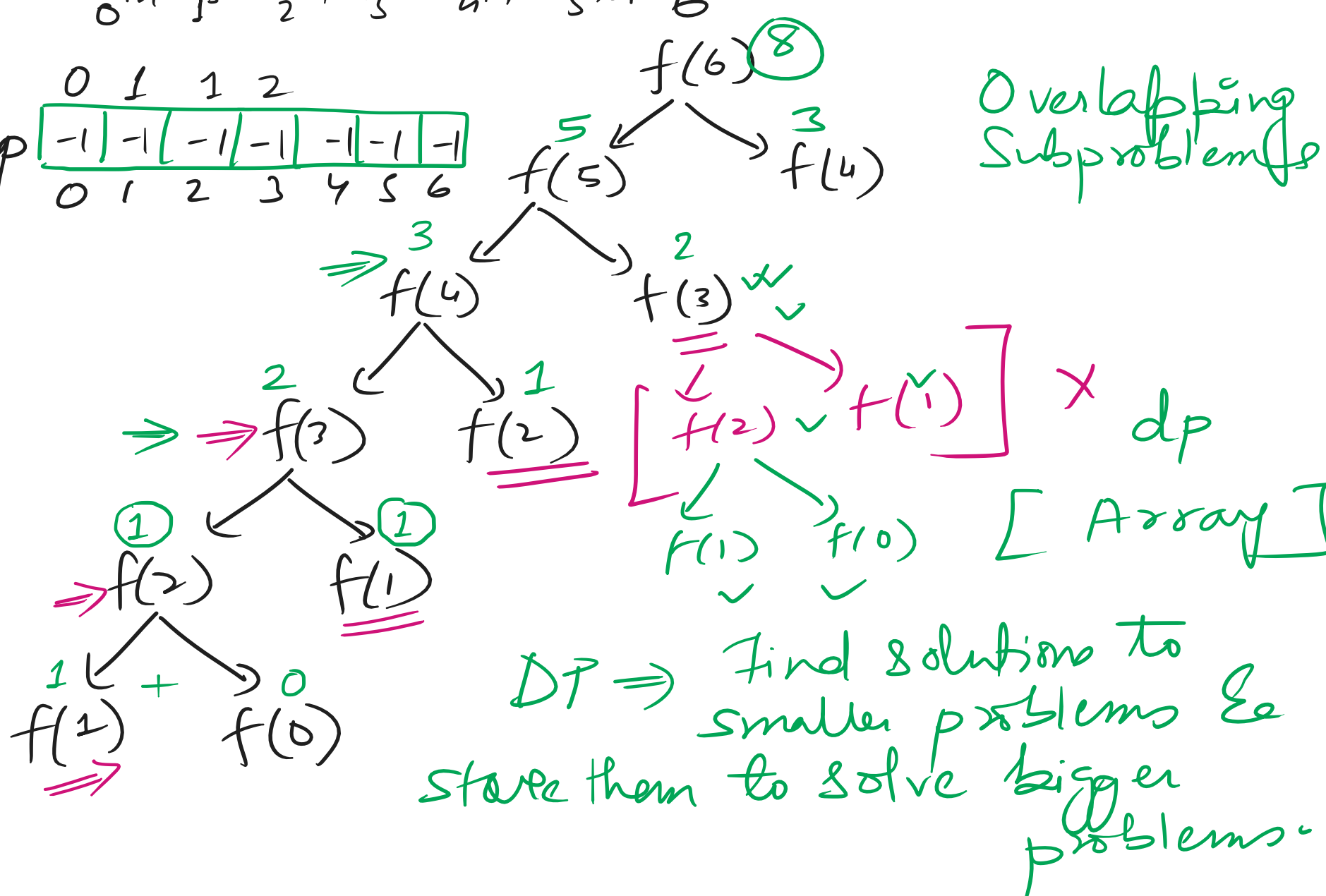
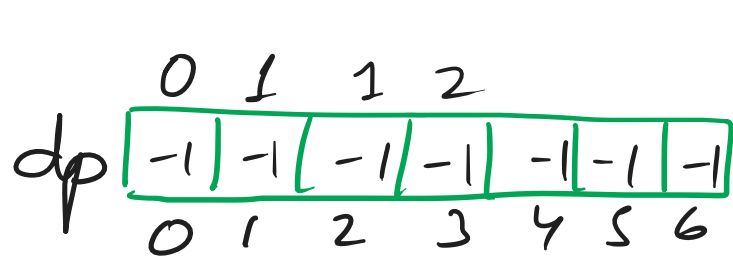
$$5 + 4 + f(123)$$

$$\begin{array}{l} 5 + 4 + 3 + f(12) \\ 5 + 4 + 3 + 2 + f(1) \end{array}$$

$$14 + 1 = 15$$

0 1 1 2 3 5 8
0th 1st 2nd 3rd 4th 5th 6th

$$f(n) = f(n-1) + f(n-2)$$



Recursive : $\begin{matrix} R & O(n) \\ A & O(n) \\ n & \rightarrow O(n) \end{matrix} \rightarrow \begin{matrix} TC \\ SC \end{matrix}$

Memo $\begin{matrix} \nearrow O(n) \\ A O(n) \rightarrow TC \\ \nwarrow dp O(n) \rightarrow SC \end{matrix}$

Tabulation: $dp[n] = [-1, -1, -1, -1, -1, -1, -1]$
 Rec. 0 1 2 3 4 5 6

~~Rec~~ X
dp $O(n)$ + ~~$O(n)$~~
 Sc $O(n)$
 Array

(1 2 3 4 5 6)
 0 1 2 3 4 5 6
 0 1 -1 -1 -1 -1 -1
 0 1 2 $\rightarrow n$
 don't know
 for (2 - 1 n)
 $dp[i] = dp[i-1] + dp[i-2]$

$O(n)$ → for array, string $[0 - (n-1)]$

Normal variables $\rightarrow O(1) \rightarrow \underline{\underline{\text{constant}}}$

0	1	2	3	4	5
0	1	1	2	3	5

```

int p2 = 0;
int p1 = 1;
for (i = 2; i <= n; i++) {
    int curr = p2 + p1;
    p1 = curr;
    p2 = p1;
}
return p1;

```

$$p_2 = 0$$
$$p_1 = 1$$

DP

$$\begin{array}{r} W \\ R \\ RM \\ RT \\ \times \underline{SO} \\ \hline \hline \end{array}$$
$$\begin{aligned} T_C &\rightarrow O(n) \\ S_C &\rightarrow O(1) \end{aligned}$$